Grafos

Objetivos:

- Presentar la estructura de datos grafo
- Mostrar la especificación e implementación del TAD Grafo
- Principales algoritmos de manipulación de grafos

Contenido

1. Introducción

1.1. Definición

1.2. Terminología fundamental

2. TAD grafo

2.1. Especificación

2.2. Implementación

Mediante listas de arcos

Mediante listas de adyacencia

Mediante matriz de adyacencia

Mediante mapas de adyacencia

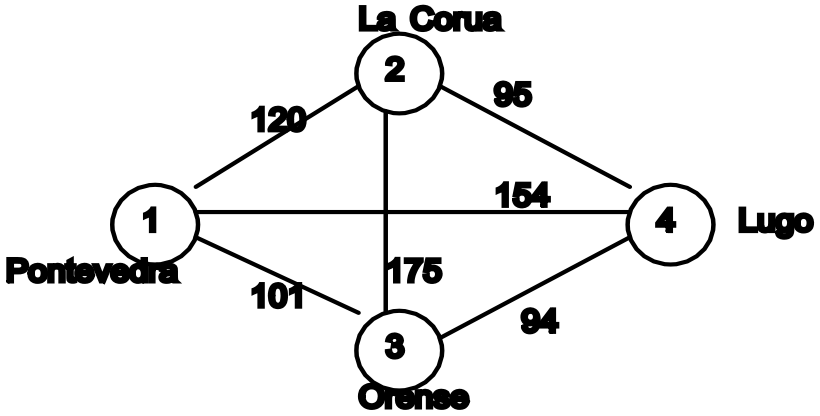
Grafos

- **Bibliografía**
  - Goodrich M., Tamassia R. y Goldwasser M., *Data structures and Algorithms in JAVA 6ª ed.* John Wiley & Sons Inc., 2015. Págs.580-592
  - Goldman S. y Goldman K. *A practical guide to data structures and algorithms using Java*. Chapman & Hall/CRC. 2008. Págs.843-856
  - Allen Weiss, M. *Estructuras de datos en java*. 4ª ed. Pearson, 2013 Págs.515-557

Definición

- Un grafo es una estructura capaz de representar relaciones complejas entre objetos de un mismo tipo. Formalmente se representa mediante el par G = (V, A), donde:
  - V es un conjunto de objetos llamados **vértices o nodos** (*Un vértice es un objeto compuesto de una etiqueta*)
  - A es un conjunto de objetos denominados **aristas o arcos.** Las aristas representan relaciones entre los vértices, de forma que una arista es un par (u, v) de vértices de V, y opcionalmente una etiqueta.

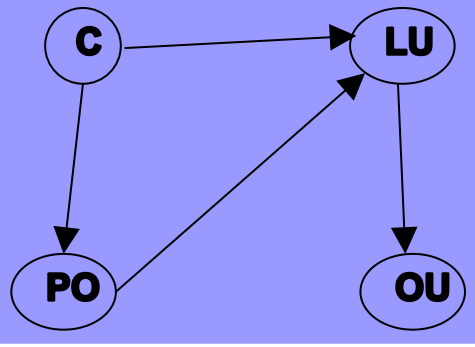
- Un ejemplo de grafo podría ser la red de carreteras de Galicia, donde los vértices representan las ciudades y las aristas la carretera que une dos ciudades:



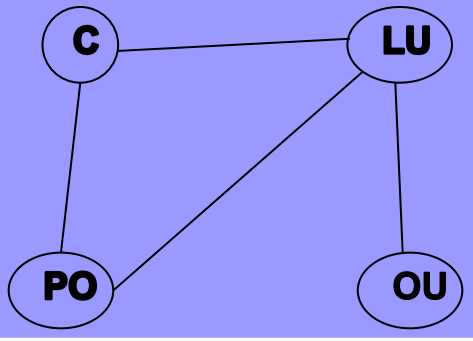
Definición

- Básicamente, podemos clasificar los grafos en 4 tipos dependiendo de dos criterios:

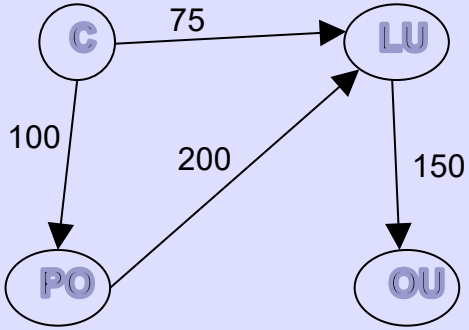
Grafo dirigido:
Las aristas forman pares ordenados



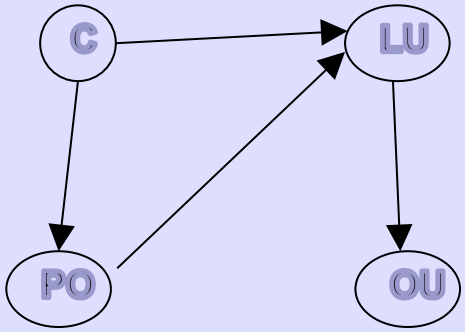
Grafo no dirigido:
Las aristas no son pares ordenados



Grafo etiquetado:
Se asocia información a cada arista:

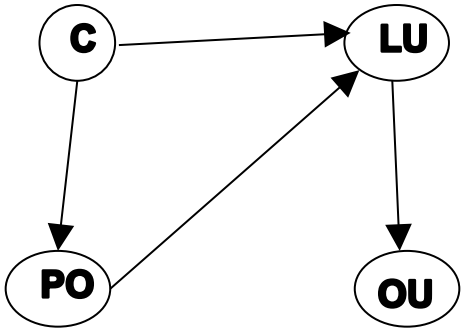


Grafo no etiquetado:
No se asocia información a cada arista



Terminología

- Algunas definiciones sobre grafos:
  - Un **camino** en un grafo G = (V, A) es una secuencia de vértices $v_1, ..., v_n \in V$, con n>=1, tal que $(v_{i-1}, v_i) \in A$ para i= 1...n-1
  - La **longitud** de un camino es su número de vértices menos 1
  - Un camino es **simple** si todos sus vértices, excepto tal vez el primero y el último, son distintos
  - Un **ciclo** es un camino simple de longitud no nula que empieza y termina en el mismo vértice



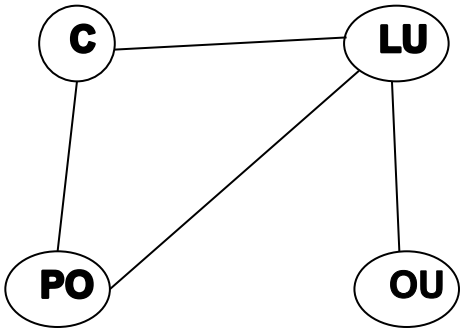
camino = {C, PO, LU, OU}

longitud(camino) = 3

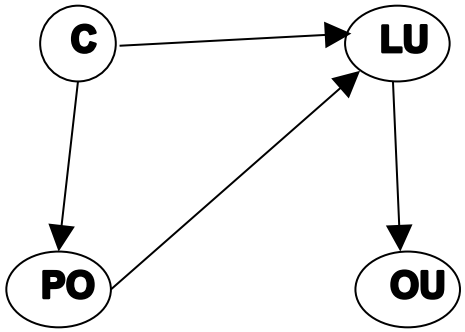
Terminología

- En un grafo no dirigido, el **grado** de un vértice *v* es el número de aristas que contiene a *v*

- En un grafo dirigido se distingue entre el **grado de entrada** y el **grado de salida**: el grado de entrada de un vértice *v* es el número de aristas que llegan a *v* (antecesores), y el grado de salida de un vértice *v* es el número de aristas que salen de *v* (sucesores).



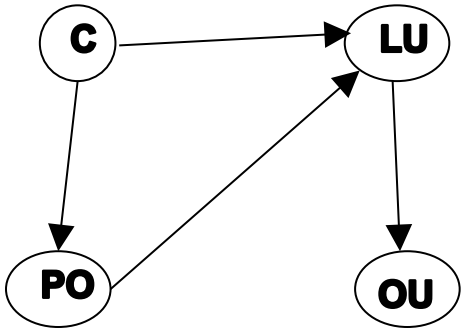
grado (**LU**) = 3



grado entrada (**LU**) = 2
grado salida (**LU**) = 1

Terminología

- Se dice que el vértice *y* es **sucesor** o **adyacente** del vértice *x* si existe una arista que tenga por origen a *x* y por destino a *y*, es decir, si la arista (x, y) $\in$ A
- Se dice que el vértice x es **antecesor** del vértice *y* si existe una arista que tenga por origen a *x* y por destino a *y*, es decir, si la arista (x, y) $\in$ A

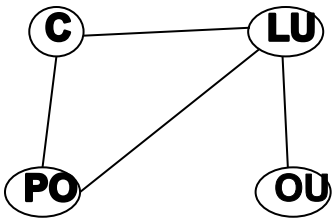


ady(LU) = {OU}

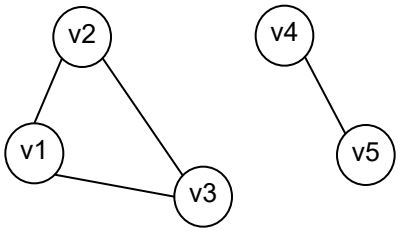
ant(LU) = {C, PO}

Terminología

- Un grafo no dirigido G es **conexo** si existe un camino entre cualquier par de nodos que forman el grafo.

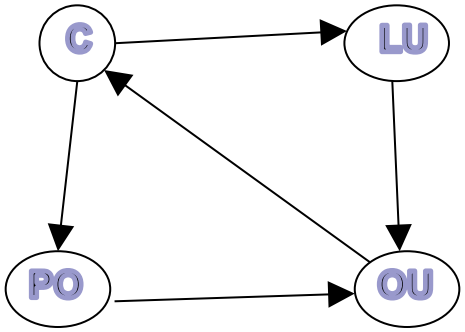


Conexo



No conexo

- Un grafo dirigido es **fuertemente conexo** si existe un camino entre cualquier par de nodos que forman el grafo



TAD Graph<T,E> - Especificación

Graph<T,E>

// Declaración de tipos: Vertex<T>, Edge<T,E>

// Características**:** Es un conjunto de vértices y un conjunto de arcos. No admite vértices null. Solo admite un arco entre los vértices. Los objetos son modificables.

public Graph( )

// *Produce*: un grafo vacío, sin vértices

public int numberOfVertices();

//*Produce*: número de vértices del grafo

public boolean containsVertex(Vertex<T> vertex);

// *Produce*: cierto si vertex está en this; falso en caso contrario

public boolean containsEdge(Vertex<T> source, Vertex<T> target, E label);

// *Produce*: cierto si el arco (vértice origen, vértice destino y etiqueta) está en this, falso en caso contrario

public Set<Vertex<T>> getVertices()

// *Produce*: devuelve un conjunto con todos los vértices del grafo

public Set<Edge<T, E>> getEdges();

// *Produce*: devuelve un conjunto con todos los arcos del grafo

public Set<Vertex<T>> getAdjacentsVertex(Vertex<T> vertex) **throws** NullPointerException

// *Produce*: si vertex es null lanza NullPointerException; en otro caso, devuelve un conjunto con los vértices adyacentes (sucesores) de vertex

TAD Graph<T,E> - Especificación

public boolean addVertex(Vertex<T> vertex) **throws** NullPointerException;

// Modifica: this

// Produce: lanza NullPointerException si vertex es null. Inserta vertex en this y devuelve cierto; si ya existía, devuelve false.

public boolean addEdge(Vertex<T> source, Vertex<T> target, E label) **throws** NullPointerException, IllegalArgumentException

// Modifica: this

// Produce: lanza NullPointerException si alguno de los vértices es null. Lanza IllegalArgumentException si alguno de los vértices no está en el grafo. Añade un arco dirigido al grafo y devuelve cierto; si ya existía un arco entre los vértices, retorna falso

public boolean removeVertex(Vertex<T> vertex) **throws** NullPointerException;

// Modifica: this

// Produce: si vertex es null, lanza NullPointerException. Elimina el vértice y todos los arcos asociados y devuelve cierto; si el vertice no existe en el grafo, devuelve falso.

public boolean removeEdge(Vertex<T> source, Vertex<T> target, E label) **throws** NullPointerException;

// Modifica: this

// Produce: lanza NullPointerException si alguno de los vértices es null. Si el arco no existe, devuelve falso; en otro caso elimina el arco y retorna cierto.

public boolean isEmpty();

//Produce: cierto si el grafo está vacío; falso en caso contrario

public void clear();

//Modifica: this

//Produce: elimina los vértices y arcos del grafo, convirtiéndolo en un grafo vacío.

Uso TAD Graph<T,E>

Ejemplo:

```
public static <T,E> int outDegree (Graph<T,E> graph, Vertex<T> vertex){
        return graph.getAdjacentsVertex(vertex).size();
}
```

```
public static <T, E> Set<Vertex<T>> getPredecessors(Graph<T, E> graph, Vertex<T> vertex){
```

Set<Vertex<T>> pred = {C, PO }

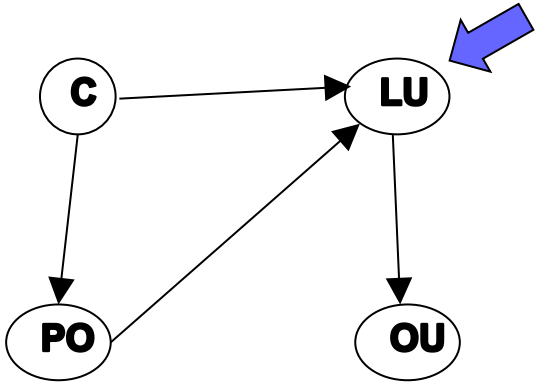
graph.getVertices = {C, LU, OU, PO}

{LU, PO}

{OU}

{}

{LU}



Uso TAD Graph<T,E>

Ejemplo:

```
public static <T,E> int outDegree (Graph<T,E> graph, Vertex<T> vertex){
        return graph.getAdjacentsVertex(vertex).size();
}
```

```
public static <T, E> Set<Vertex<T>> getPredecessors(Graph<T, E> graph, Vertex<T> vertex){
        Set<Vertex<T>> setOfPred = new HashSet<>();
         for (Vertex<T> w : graph.getVertices()) {
             Set<Vertex<T>> setOfAdy = graph.getAdjacentsVertex(w);
             if (setOfAdy.contains(vertex)) {
               setOfPred.add(w);
             }
          }
           return setOfPred;
        }
```

Algoritmos de recorrido

- La operación de recorrer un grafo consiste en partir de un vértice determinado y visitar todos aquellos vértices que son accesibles desde él en un determinado orden.
- Dos estrategias:
  - recorrido en profundidad (*DFS: Depth First Search*)
  - recorrido en anchura (*BFS: Breadh First Search*).

Algoritmos de recorrido

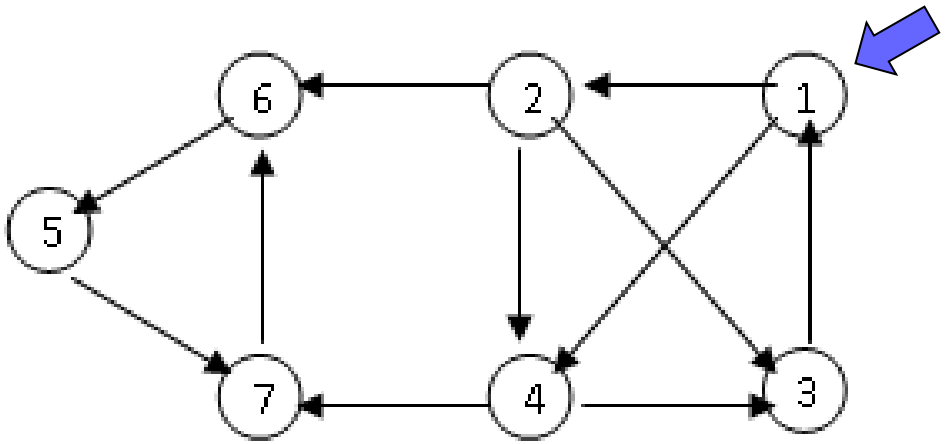
Recorrido en profundidad

- Es una generalización del recorrido en *preorden* de un árbol.
- Pasos:
  1. Se visita el vértice del que se parte, **v**.
  2. Se selecciona un vértice **w**, adyacente a **v** y que todavía no se haya visitado.
  3. Se realiza el recorrido en profundidad partiendo del vértice **w**.
  4. Cuando se encuentra un vértice cuyo conjunto de adyacentes han sido visitados en su totalidad se retrocede hasta el último vértice visitado que tenga vértices adyacentes no visitados y se ejecuta el paso 2.
- Suponemos la existencia de un conjunto (*visitados*) para ir almacenando los vértices del grafo por los que se va pasando.
- En principio el conjunto de vértices visitados estará vacío.

Algoritmos de recorrido

visitados = {1, 2, 3, 4, 7, 6, 5 }

g.adyacentes(1) = {2, ~~4~~}
g.adyacentes(2) = {3, 4, ~~5~~}
g.adyacentes(3) = {~~1~~}
g.adyacentes(4) = {~~3~~, 7}
g.adyacentes(7) = {6}
g.adyacentes(6) = {5}
g.adyacentes(5) = {~~7~~}



Recorrido en profundidad: **1 2 3 4 7 6 5**

Algoritmos de recorrido

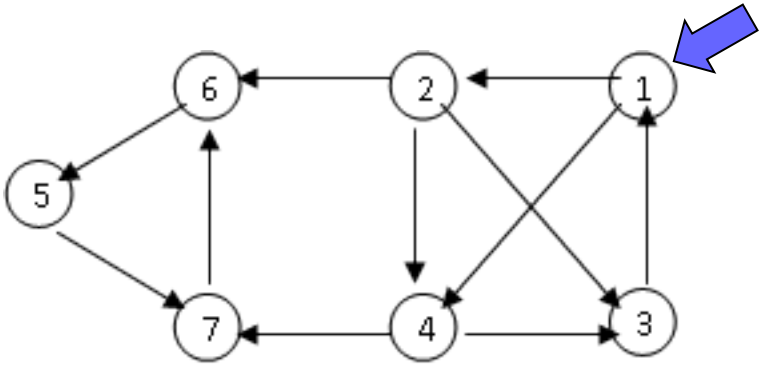
```
public static <T,E> void dfsRecursive (Graph<T,E> graph, Vertex<T> vertex) {
        Set<Vertex<T>> visited = new HashSet<>();
        dfs(graph, vertex, visited);
}

private static <T,E> void dfs(Graph<T,E> graph, Vertex<T> vertex,
                                                Set<Vertex<T>> visited) {
        System.out.println(vertex);
        visited.add(vertex);
        for(Vertex<T> adj : graph.getAdjacentsVertex(vertex)){
            if (!visited.contains(adj))
              dfs(graph, adj, visited);
        }
}
```

Algoritmos de recorrido

Recorrido en anchura

- Generaliza el recorrido en *anchura* (por niveles) de un árbol.
- Pasos:
  1. Se visita el vértice del que se parte, **v**.
  2. Se visitan todos sus adyacentes que no estuvieran ya visitados, y así sucesivamente. Esto es, se visitan todos los vértices adyacentes antes de pasar a otro vértice.
- Utilizamos un conjunto (*visitados*) para ir almacenando los vértices del grafo por los que se va pasando.
- En una cola (*porExplorar*) se mantienen los vértices adyacentes que se han obtenido a partir de los vértices visitados y cuyos adyacentes aún restan por explorar.
- Inicialmente, tanto el conjunto de vértices visitados como la cola de vértices por visitar estarán vacíos.

Algoritmos de recorrido



visitados = {1, 2, 4, 3, 6, 7, 5 }

porExplorar = {~~1~~, ~~2~~, ~~4~~, ~~3~~, ~~6~~, ~~7~~, ~~5~~ }

g.adyacentes(1) = {2, 4}

g.adyacentes(2) = {3, ~~4~~, 6}

g.adyacentes(4) = {~~3~~, 7}

g.adyacentes(3) = {~~1~~}

g.adyacentes(6) = {5}

g.adyacentes(7) = {~~6~~}

g.adyacentes(5) = {~~7~~}

Recorrido en anchura: **1 2 4 3 6 7 5**

Algoritmos de recorrido

```
public static <T,E> void bfs(Graph<T,E> graph, Vertex<T> vertex){
    Set<Vertex<T>> visited = new HashSet<>();
    Queue<Vertex<T>> toExplore = new LinkedQueue<>();

    toExplore.add(vertex);
    visited.add(vertex);

    do {
        vertex = toExplore.remove();
        System.out.println(vertex);
        for(Vertex<T> adj : graph.getAdjacentsVertex(vertex)) {
            if (! visited.contains(adj)) {
                toExplore.add(adj);
                visited.add(adj);
            }
        }
    }
    while (! toExplore.isEmpty());
}
```